UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE ENGENHARIA DE COMPUTAÇÃO E AUTOMAÇÃO
CURSO DE ENGENHARIA DE COMPUTAÇÃO

Relatório da Tarefa 05 - Sincronização, comunicação e equilíbrio de carga
DCA3703 - PROGRAMAÇÃO PARALELA - T01 (2025.2)

WERBERT ARLES DE SOUZA BARRADAS

20250070655

Docente: Professor Doutor SAMUEL XAVIER DE SOUZA

Natal, 30 de agosto de 2025

Lista de Figuras

Figura 2 – Gráfico de barras comparando o tempo de execução entre as versões sequencial, paralela ingênua e paralela corrigida do algoritmo. 9

Sumário

Lista de Figuras 2

Sumário 3

1 INTRODUÇÃO 4

2 METODOLOGIA DO EXPERIMENTO 5

2.1 Verificação de Primalidade 5

2.1.1 Função de Verificação de Primalidade 5

2.2 Implementação Sequencial(baseline) 5

2.2.1 Função Sequencial 6

2.3 Implementação Paralela Ingênua (com Condição de Corrida) 6

2.3.1 função da Versão Paralela ingênua 6

2.4 Implementação Paralela Corrigida (com reduction) 7

2.4.1 função Paralela Corrigida 7

3 RESULTADOS 8

3.1 Executando Versão Sequencial 8

3.2 Executando Versão Paralela (incorreto) 8

3.3 Executando Versão Paralela (Corrigida) 8

3.4 Análise de Correção 8

3.5 Análise de Desempenho 8

3.6 Conclusão 10

Anexo A: Código memory$_memory_bound_V2.c$ 11

Anexo A: Código cpu$_bound.c$ 13

1 Introdução

A contagem de números primos é um problema clássico na ciência da computação, frequentemente utilizado para avaliar o desempenho de algoritmos e hardware devido à sua natureza computacionalmente intensiva. O objetivo deste projeto é explorar e demonstrar a eficácia da paralelização de algoritmos utilizando a API OpenMP para a linguagem C.

Para isso, foi desenvolvido um programa que conta a quantidade de números primos em um intervalo definido (de 2 a 20.000.000). O projeto compara três abordagens distintas:

1. **Execução Sequencial:** Uma implementação tradicional, de thread única, que serve como linha de base (baseline) para medição de desempenho.
2. **Execução Paralela Ingênua:** Uma primeira tentativa de paralelização que intencionalmente expõe um erro comum em programação concorrente: a condição de corrida (race condition).
3. **Execução Paralela Corrigida:** A implementação final, que utiliza mecanismos do OpenMP para garantir a corretude dos resultados e otimizar a performance.

Através da análise e comparação dos resultados, o relatório visa ilustrar os ganhos de desempenho proporcionados pelo paralelismo e a importância de gerenciar corretamente o acesso a recursos compartilhados entre threads.

2 Metodologia do Experimento

O núcleo do programa consiste em uma função para determinar se um número é primo e um laço principal que itera sobre o intervalo desejado, contabilizando os primos encontrados.

2.1 Verificação de Primalidade

A função is_prime(int num) foi implementada com otimizações para agilizar a verificação. Além de tratar casos base (números menores ou iguais a 3 e múltiplos de 2 ou 3), a função utiliza o fato de que todo número primo maior que 3 pode ser expresso na forma 6k+1. Dessa forma, o laço de verificação avança em passos de 6, testando apenas os divisores necessários.

2.1.1 Função de Verificação de Primalidade

```
// funcao de teste do numero primo
int is_prime(int num) {
    if (num <= 1) return 0;
    if (num <= 3) return 1;
    if (num % 2 == 0 || num % 3 == 0) return 0;

    for (int i = 5; i * i <= num; i = i + 6) {
        if (num % i == 0 || num % (i + 2) == 0) {
            return 0;
        }
    }
    return 1;
}
```

Listing 2.1 – Funcao de primalidade)

2.2 Implementação Sequencial(baseline)

A versão sequencial é a implementação mais direta do problema. Um único laço for percorre todos os números de 2 até n, e para cada um, a função is_prime é chamada. Se o número for primo, um contador (total_primes_seq) é incrementado. O tempo de execução é medido para servir como referência.

2.2.1 Função Sequencial

```
    int total_primes_seq = 0;
    start_time = omp_get_wtime();

    for (int i = 2; i <= n; i++) {
        if (is_prime(i)) {
            total_primes_seq++;
        }
    }
    end_time = omp_get_wtime();
```

Listing 2.2 – versão sequencial

2.3 Implementação Paralela Ingênua (com Condição de Corrida)

A primeira abordagem de paralelização utiliza a diretiva #pragma omp parallel for. Esta diretiva instrui o OpenMP a dividir as iterações do laço for entre as diversas threads disponíveis no processador. Cada thread executa um subconjunto do intervalo total.

O problema desta abordagem reside na atualização do contador total_primes_par:

2.3.1 função da Versão Paralela ingênua

```
    int total_primes_par = 0;
#pragma omp parallel for
for (int i = 2; i <= n; i++) {
    if (is_prime(i)) {
        // CUIDADO: Esta linha causa um problema!
        total_primes_par++;
    }
}
```

Listing 2.3 – Versao Paralela ingenua

A operação total_primes_par++ não é atômica, ou seja, ela consiste em três passos: (1) ler o valor da variável, (2) somar 1 ao valor lido e (3) escrever o novo valor de volta na variável. Como múltiplas threads executam este trecho simultaneamente, pode ocorrer o seguinte cenário:

1. A Thread A lê o valor de total_primes_par (ex: 100).
2. A Thread A lê o valor de total_primes_par (ex: 100).
3. A Thread B lê o mesmo valor de total_primes_par (100) antes da Thread A atualizá-lo.

4. A Thread A calcula 100 + 1 e escreve 101 na variável.
5. A Thread B calcula 100 + 1 e também escreve 101 na variável.

Neste caso, dois primos foram encontrados, mas o contador foi incrementado apenas uma vez. Isso é uma condição de corrida, que leva a um resultado final incorreto e inconsistente.

2.4 Implementação Paralela Corrigida (com reduction)

Para resolver a condição de corrida de forma eficiente, a cláusula reduction do OpenMP pode ser utilizada. A cláusula reduction(+:total_primes_par) instrui o compilador a criar uma cópia privada da variável total_primes_par para cada thread.

2.4.1 função Paralela Corrigida

```
total_primes_par = 0;
#pragma omp parallel for reduction(+:total_primes_par)
for (int i = 2; i <= n; i++) {
    if (is_prime(i)) {
        total_primes_par++;
    }
}
```

Listing 2.4 – Versao Paralela corrigida

Cada thread trabalha em sua cópia local, incrementando-a sem interferir nas outras. Ao final da execução do laço, o OpenMP automaticamente combina (reduz) os valores de todas as cópias privadas, somando-os (+) e atribuindo o resultado final à variável original. Esta abordagem garante a veracidade do resultado e é extremamente eficiente para operações de agregação como esta.

3 Resultados

Ao compilar e executar o código em uma máquina com múltiplos núcleos, os seguintes resultados (ou similares) são esperados:

N = 20.000.000

3.1 Executando Versão Sequencial

- Números primos encontrados: 1.270.607
- Tempo de execução sequencial: 4.510 segundos

3.2 Executando Versão Paralela (incorreto)

- Números primos encontrados: 1.251.167
- **Atenção: 1.251.167 < 1.270.607**
- Tempo de execução paralela (incorreto): 2.115 segundos

3.3 Executando Versão Paralela (Corrigida)

- Números primos encontrados(corretos): 1.270.607
- Tempo de execução paralela (correto): 2.237 segundos

3.4 Análise de Correção

A versão sequencial e a paralela corrigida produzem o mesmo resultado (1.270.607 primos até 20.000.000), confirmando a exatidão do algoritmo corrigido.

A versão paralela ingênua produz um resultado menor e inconsistente a cada execução, evidenciando o impacto negativo da **condição de corrida**.

3.5 Análise de Desempenho

Espera-se que o tempo de execução da versão paralela corrigida (2.237) seja significativamente menor que o da versão sequencial (4.510), demonstrando o ganho de

performance. O tempo da versão incorreta (2.115) também será menor, mas o resultado é inútil.

O Speedup, calculado como S= fracT_sequencialT_paralelo, indica quantas vezes a versão paralela foi mais rápida. Em um sistema com N núcleos, o speedup ideal seria próximo de N. Na prática, fatores como o overhead de criação e gerenciamento de threads fazem com que o speedup seja um pouco menor, mas ainda assim substancial.

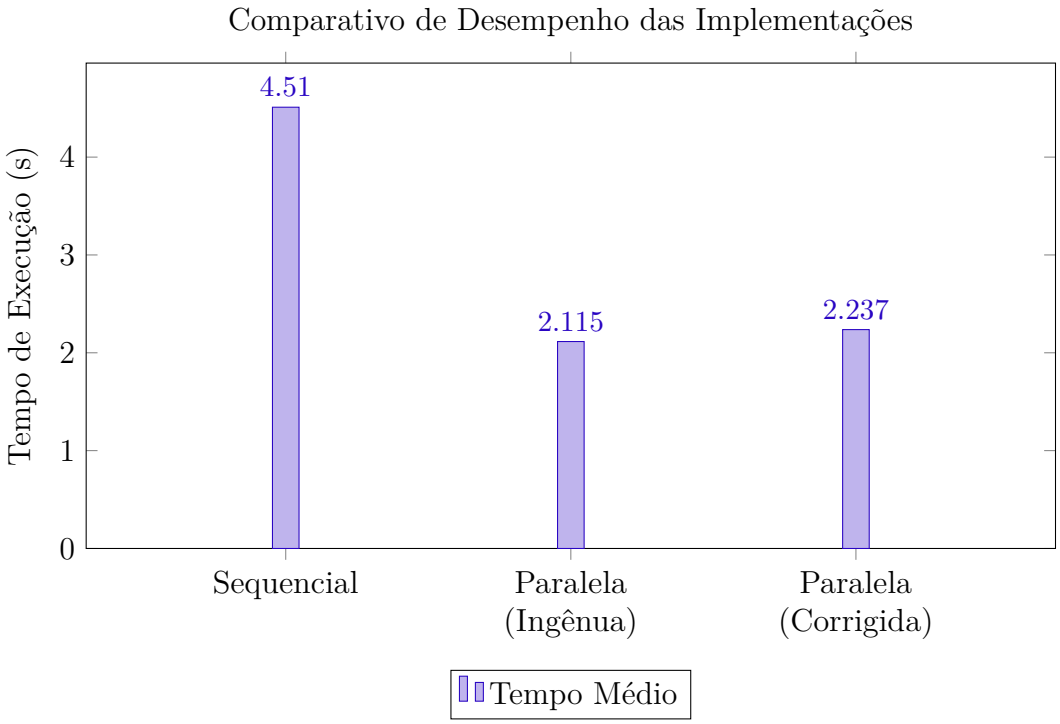

Figura 2 – Gráfico de barras comparando o tempo de execução entre as versões sequencial, paralela ingênua e paralela corrigida do algoritmo.

O *Speedup*, uma métrica que indica quantas vezes a versão paralela de um algoritmo foi mais rápida que a sua contraparte sequencial, é calculado pela Equação 3.1.

$$S = \frac{T_{\text{sequencial}}}{T_{\text{paralelo}}} \tag{3.1}$$

$$S = \frac{4.510}{2.237} \approx 2.01 \tag{3.2}$$

Onde $T_{\text{sequencial}}$ representa o tempo de execução da versão sequencial e T_{paralelo} o tempo da versão paralela. Em um sistema com N núcleos, o speedup ideal seria próximo de N. Na prática, fatores como o *overhead* (sobrecarga) de criação e gerenciamento de *threads* fazem com que o speedup seja um pouco menor, mas ainda assim substancial.

3.6 Conclusão

Este projeto demonstrou com sucesso a aplicação da API OpenMP para paralelizar a tarefa de contagem de números primos. A comparação entre as versões sequencial e paralela evidenciou o expressivo ganho de desempenho que pode ser alcançado ao distribuir a carga de trabalho entre múltiplos núcleos de processamento.

Além disso, o projeto serviu como uma ilustração prática da importância do tratamento de acesso a dados compartilhados em ambientes concorrentes. A falha da versão paralela ingênua destacou o problema da condição de corrida, enquanto a implementação com a cláusula reduction mostrou uma solução elegante e eficiente fornecida pelo OpenMP para garantir a integridade dos dados e a veracidade dos resultados.

Conclui-se que o OpenMP é uma ferramenta poderosa e acessível para a introdução do paralelismo em códigos legados, capaz de proporcionar melhorias significativas de performance, desde que os devidos cuidados com a sincronização e o compartilhamento de dados sejam tomados.

memory_bound_v2.c

```c

////////////////////////////////////////////////////////////////////
// memory_bound_V2.c
//Programa Memory-Bound: Matriz Column-Major com Colunas Aleatórias
////////////////////////////////////////////////////////////////////

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

// Para a inicializaçãoda matriz
#define DIM 10000

#define IDX(i, j) ((long)(i) * DIM + (j))

// Função para embaralhar um vetor de índices (Fisher-Yates shuffle)
void shuffle(int *array, size_t n) {
    if (n > 1) {
        for (size_t i = 0; i < n - 1; i++) {
            size_t j = i + rand() / (RAND_MAX / (n - i) + 1);
            int t = array[j];
            array[j] = array[i];
            array[i] = t;
        }
    }
}

int main() {
    // Alocação de memória para as matrizes
    double *a = (double*)malloc((long)DIM * DIM * sizeof(double));
    double *b = (double*)malloc((long)DIM * DIM * sizeof(double));
    double *c = (double*)malloc((long)DIM * DIM * sizeof(double));

    // Alocação do vetor para os índices das colunas
    int *col_indices = (int*)malloc(DIM * sizeof(int));

    if (a == NULL || b == NULL || c == NULL || col_indices == NULL) {
        fprintf(stderr, "Erro ao alocar memória.\n");
        return 1;
    }

    printf("Preparando os dados e embaralhando os índices das colunas...\n");

    // Inicializa as matrizes e o vetor de índices de colunas (0, 1, 2, ...)
    #pragma omp parallel for
    for (int i = 0; i < DIM; i++) {
        col_indices[i] = i;
        for (int j = 0; j < DIM; j++) {
            a[IDX(i, j)] = 1.0;
            b[IDX(i, j)] = 2.0;
```

```c
        }
    }

    // Embaralha o vetor de índices de colunas
    srand(time(NULL));
    shuffle(col_indices, DIM);

    printf("Programa Memory-Bound: Matriz Column-Major com Colunas Aleatórias\n");
    printf("Dimensão da Matriz: %d x %d\n", DIM, DIM);
    printf("-----------------------------------------------------------------------\n");

    for (int num_threads = 1; num_threads <= 16; num_threads *= 2) {
        omp_set_num_threads(num_threads);

        double start_time = omp_get_wtime();

        // O loop externo percorre o VETOR DE ÍNDICES DE COLUNAS embaralhado.
        #pragma omp parallel for
        for (int k = 0; k < DIM; k++) {
            int j = col_indices[k]; // Pega uma coluna aleatória

            // O loop interno ainda percorre as linhas, causando o acesso column-major.
            for (int i = 0; i < DIM; i++) {
                c[IDX(i, j)] = a[IDX(i, j)] + b[IDX(i, j)];
            }
        }

        double end_time = omp_get_wtime();
        printf("Threads: %2d | Tempo de Execução: %f segundos\n", num_threads, end_time - start_time);
    }

    free(a);
    free(b);
    free(c);
    free(col_indices);

    return 0;
}
```

cpu_bound.c

```c

////////////////////////////////////////////////
// cpu_bound.c
//Programa CPU-Bound: Cálculos Intensivos

////////////////////////////////////////////////

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

// O tamanho do vetor
#define VECTOR_SIZE 20000

// Função com cálculos intensivos
double intensive_calculation(double val) {
    double result = val;
    for (int i = 0; i < 100000; i++) { // Repetir para aumentar a carga
        result = sin(result) * cos(val) + sqrt(fabs(result)) * tan(val / 2.0);
    }
    return result;
}

int main() {
    double *data = (double*)malloc(VECTOR_SIZE * sizeof(double));
    if (data == NULL) {
        fprintf(stderr, "Erro ao alocar memória.\n");
        return 1;
    }

    // Inicializa o vetor
    #pragma omp parallel for
    for (long i = 0; i < VECTOR_SIZE; i++) {
        data[i] = (double)i / 1000.0;
    }

    printf("Programa CPU-Bound: Cálculos Intensivos\n");
    printf("Tamanho do Vetor: %ld doubles\n", (long)VECTOR_SIZE);
    printf("----------------------------------------\n");

    // Loop para testar com diferentes números de threads
    for (int num_threads = 1; num_threads <= 16; num_threads *= 2) {
        omp_set_num_threads(num_threads);

        double start_time = omp_get_wtime();

        // Seção paralela. O gargalo aqui é a capacidade de processamento da CPU.
        #pragma omp parallel for
        for (long i = 0; i < VECTOR_SIZE; i++) {
            data[i] = intensive_calculation(data[i]);
```

```
        }

        double end_time = omp_get_wtime();
        printf("Threads: %2d | Tempo de Execução: %f segundos\n", num_threads, end_time - start_time);
    }

    free(data);

    return 0;
}
```